Software Requirements Specification

for

EagleEye Parking

Version 1.0

Prepared by
Nhan Vu
Thuan Nguyen
Alperen Dalkir
Luis Lozano
Aaditya Yadav

UNT- CSCE4901.502- Capstone1- Group 3

February 4, 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction....3
1.1 Purpose....3
1.2 Intended Audience and Reading Suggestions....3
1.3 Product Scope....3
1.4 References....3
2. Overall Description....4
2.1 Product Perspective....4
2.2 Product Functions....4
2.3 User Classes and Characteristics....4
2.4 Operating Environment....5
2.5 Design and Implementation Constraints....5
2.6 User Documentation....6
2.7 Assumptions and Dependencies....6
3. External Interface Requirements....8
3.1 User Interfaces....8
Mobile Application Interface (Student/End-User)....8
Administration Dashboard (Web-Based)....8
Error Message & Feedback Standards....8
3.2 Hardware Interfaces....9
1. Mobile Device Sensors (Direct Interface)....9
2. LPR Patrol Vehicles & AIMS Infrastructure (Indirect Interface)....9
3. Network Infrastructure....9
3.3 Software Interfaces....10
1. ParkZen Integration (SDK & Cloud API)....10
2. UNT AIMS Integration (Enforcement Data)....10
3. Database Management (PostgreSQL)....10
4. Push Notification Services (APNS & FCM)....10
5. Integrated Web Views (AIMS Permit Portal)....11
3.4 Communications Interfaces....11
1. Network Protocols and Standards....11
2. Mobile-to-Server Communication....11
3. Server-to-External API Communication....11
4. Electronic Forms & User Input....11
4. System Features....13
4.1 Real-Time Parking Occupancy Display....13
4.2 AIMS/LPR Data Integration....15
4.3 Parking Inventory Management....15
4.4 Push Notifications and Alerts....17
4.5 Hazard and Closure Reporting....18

4.6 Permit Information and Park-Down Rules Display....19
4.7 Administrative Dashboard and Analytics....20
4.8 Permit Purchase Integration....21
5. Other Nonfunctional Requirements....22
5.1 Performance Requirements....22
5.2 Safety Requirements....22
5.3 Security Requirements....22
5.4 Software Quality Attributes....23
5.5 Business Rules....23
6. Other Requirements....24
6.1 Database Requirements....24
6.2 Legal and Reuse Requirements....24
1. Entity-Relationship Diagram (ERD)....25
2. Data Dictionary (subject to change)....25
3. Data Flow Diagram (DFD)....25

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the UNT Parking Availability feature to be integrated into the existing UNT Go mobile application. This document provides a detailed description of the system's intended capabilities, constraints, assumptions, and interactions with external systems. The project aims to deliver a real-time, map-based parking availability solution that improves the parking experience for students, faculty, staff, and visitors by reducing search time and campus congestion.

1.2 Intended Audience and Reading Suggestions

This document is intended for:

- **Jim Coffey** (Project Sponsor)
- **Keitha Pearce** (Executive Director, IT Support)
- **Manos Chatzopoulos** (ParkZen CEO & Co-Founder)
- UNT Transportation Service and the UNT IT department
- Development Team (Capstone Group 3)

Readers interested in system behavior should focus on the **Product Functions** and **Overall Description** sections. Developers and testers should refer to the **Design Constraints**, **Operating Environment,** and **Assumptions and Dependencies** sections.

1.3 Product Scope

The Parking Availability feature will:

- Integrate ParkZen-style functionality into the UNT Go app
- Display real-time and predictive parking availability
- Provide an interactive, map-based interface
- Retrieve and process parking data from multiple sources (AIMS, LPR/Genetec, Premium Parking, ParkZen SDK)
- Improve campus traffic flow and user satisfaction

1.4 References

- Project Charter for UNT Parking Availability App, version 1.0
- ParkZen SDK documentation
- UNT Go technical documentation

2. Overall Description

2.1 Product Perspective

The UNT Parking Occupancy App is an aggregate function of the already established UNTGO app, designed to provide real-time occupancy information to users about the parking lots around UNT.

The app does not replace an existing system, instead it adds to an existing app and builds on some existing systems, such as the existing parking permits database and the school's license plate reading truck, which will be implemented in the coming fall. The app serves as a parking decision system that will eventually integrate permits and premium parking spots.

The app is intended to make parking at UNT faster and lessen the luck of the draw, reduce congestion from certain popular parking lots by showing users other available options, and enhance parking for UNT commuters.

2.2 Product Functions

The app's major functions will depend on whether the user is an admin or not
NON-Admin features

- View a map of campus lots and their prospective occupancy indicators
- View key stats of parking lots, such as when the data was last updated, possible hazards, and what permits are allowed in said parking lot
- Direction/navigation to the parking lot
- Data freshness for occupancy updates

Admin features

- Update, remove, and update lots
- Mark lots as close, unavailable, or hazardous
- Monitor data
- Send notifications about events/hazards to users

App functions

- Collect and show occupancy values
- Serve api data to the frontend app
- Log errors and system events

2.3 User Classes and Characteristics

The app will be used by multiple user classes with different access levels based on who they are. The main user class will be campus commuters(students, faculty, and visitors). Then there will be an admin class, which will be the ones to operate the app and make changes to the functionality and other tweaks.

End users include students, faculty, staff, and campus visitors who use the app to find parking. They will be able to..

- View parking lot occupancy levels
- Filter lots by permit type or lot rules
- Sort lots by availability or category
- View lot details(peak times and events) and data freshness

Their expertise will be similar to using any other navigation app, so it is a low expertise app. They must be familiar with smartphones and using applications. They will hopefully use the app daily to gauge their parking decisions, with spring and fall semesters starting to be the most used. Their privilege level will include access to seeing parking data and setting their permit types. This class is the most important as they will be the main users and the app must provide ease of use, clarity with prompts, and be reliable. Administrative users will be UNT parking administrators who will have access to all features in the app, while guaranteeing accuracy and reliability. Admin users will…

- Add, edit, and remove parking lot records
- Update occupancy values(manual or automated)
- Mark lots as close or partially closed
- Monitor data freshness and system warnings
- Review logs

Admin users are expected to have a higher technical expertise and may require training to be able to navigate the app features. Their use will hopefully not be as constant as that of end users, as we hope the app will seamlessly run with little need for manual input. Most use would come with sending notifications during events and or to alert of hazardous conditions. Their security access will be the highest, with access to modify data and system configurations. Second-highest priority, as they will be the ones running the app and handling errors.

System Integrators/Developers
The last class of users will be the system maintainers and developers

- Full access to the app
- Full access to app development and configuration
- Ability to modify the app and update systems
- Handle errors and fix issues

This class will hopefully maintain the app and will need to have a high level of technical expertise in coding and database handling. They will have the highest security level as they have access to the backend of the app, and they have the lowest priority as they are the ones pushing said app to end users.

2.4 Operating Environment

The Parking occupancy app will operate in a client-server architecture consisting of a front-end application, a backend that handles API calls, and a database system. The app will be designed to run off the school's existing IT infrastructure and must coexist with the UNT Go app. Backend will use Express.js and Node.js. Database:tbd. The frontend will use React and css. The app is to be deployed on Android and iOS. Hosted within a university network or a cloud provider such as AWS or Azure. Must handle peak usage times (7:30 am-10:30 am & 2:00 pm-4:00 pm)

2.5 Design and Implementation Constraints

Some of the design and implementation constraints are going to be:

Hosting and Infrastructure

- Cloud Provider or University in-house hosting

Tech Stack (subject to change)

- Language: JavaScript, Python
- Backend Framework: FashAPI
- Database: PostgreSQL
- Frontend Framework: React

Integration

- Must use Parkzen SDK
- Must align with UNT Go design language
- University SSO for permits(Duo app or the new Cisco System)
- Must follow UNT IT policies

- FERPA(when permits are deployed)
- All API endpoints must be protected with authentication
- Role-based access to prevent malicious actors
- Follow coding standards

2.6 User Documentation

User manual

- Overview of the system and features
- Description of user roles
- Instructions for viewing parking, filtering and sorting lots, viewing details, navigating lots, and administrative functions.
- Index for indicators(colors, percentages, types of lots)
- Troubleshooting
- Contact and technical support

Online Help

- FAQ
- Tooltips
- Error message explanations

Technical Documentation

- System Architecture Overview
- Database schema diagram
- API endpoint documentation
- Deployment Instructions
- Configuration settings
- Backup and recovery procedures

2.7 Assumptions and Dependencies

A1. User Hardware: Users possess smartphones with functional GPS, Accelerometer, and Bluetooth sensors running iOS 15+ or Android 10+.
A2. Data Availability: High-fidelity, real-time data feeds from the AIMS LPR system will be accessible by the middle of the Spring 2026 semester.
A3. Stakeholder Access: Stakeholders (Jim, Keitha, Manos) will provide feedback within 5-7 business days to prevent "Schedule Slip."
A4. Hand-off Hosting: UNT will provide a production environment (ModoLabs or internal server) for final deployment; otherwise, the project deliverable is a functional prototype.
A5. Infrastructure Stability: Existing UNT network APIs and authentication protocols will remain consistent throughout the development lifecycle.
A6. Policy Alignment: It is assumed that the transition to **100% Digital Permits in June 2026** will proceed as planned, as this is the primary driver for app adoption.

D1. ParkZen SDK/API: The system relies on the ParkZen backend for sensor processing and occupancy logic. Any downtime on their end will cause system-wide data loss.
D2. Tech Stack: Continued compatibility of the **FastAPI (Python)** backend, React frontend, and the ParkZen Python-based infrastructure.
D3. Network & Hosting: Availability of university-approved hosting (Azure/AWS or on-premise) that supports **Uvicorn/Gunicorn** server environments and meets UNT security standards.
D4. Development Tools: Continuous access to GitHub for version control and ModoLabs documentation for UI design.
D5. API Documentation: Successful integration is 100% dependent on receiving the **AIMS API schema and authentication keys** from UNT Parking Services.
D6. Third-Party Licensing: Implementation depends on the university maintaining its contractual relationship with ModoLabs and ParkZen.

3. External Interface Requirements

3.1 User Interfaces

Mobile Application Interface (Student/End-User)

The primary interface for EagleEye Parking is the EagleEye Parking System, integrated as a tile within the UNT Go application.

- **GUI Standards:** The interface will strictly follow the ModoLabs (UNT Go) Design System and UNT Branding Guidelines (Green/White/Black). Component layouts will follow iOS Human Interface Guidelines and Android Material Design principles.
- **Screen Layout & Constraints:**
    - **Map View:** A full-screen interactive Mapbox or Google Maps overlay centered on the UNT Denton campus.
    - **Occupancy Indicators:** To help drivers make quick decisions, parking lots are displayed as dynamic, color-coded polygons based on their current fill-rate:
        - **Green (0% – 85% full):** Plenty of parking available; recommended destination.
        - **Orange (85% – 95% full):** Limited availability; the lot is nearing capacity.
        - **Red (95%+ full or Closed):** Lot is at or over capacity; drivers should seek alternative parking.
    - **Information Drawer:** A "bottom sheet" that slides up when a lot is tapped, displaying the Lot Name, Permit Logic (e.g., "Eagle Only"), and a Data Freshness Timestamp.
- **Standard Functions:**
    - **Filter Toggle:** A persistent button to show/hide lots based on the user's saved permit type.
    - **Navigation Link:** A "Directions" button that deep-links to external navigation apps (Apple/Google Maps).
    - **Help/Legend:** A standard info icon explaining the color codes and sensor data methodology.

Administration Dashboard (Web-Based)

The ParkZen Admin Dashboard serves as the command center for UNT Parking Services staff.

- **Logical Characteristics:** A responsive, multi-pane web interface accessible via secure browser login (SSO).
- **Key Software Components:**
    - **Inventory Manager:** A table-view interface for editing space counts and permit types.
    - **Alert Composer:** A form-based tool for broadcasting push notifications (game day, towing, ice hazards).
    - **Analytics Panel:** Visual data charts (Bar/Line graphs) showing occupancy trends over time.
- **Role-Based Access Control (RBAC):** UI elements for "Lot Configuration" will be hidden from users with "View-Only" permissions to prevent unauthorized scope changes.

Error Message & Feedback Standards

- **Connectivity Alerts:** If the user's device loses internet access, a high-contrast banner will appear: *"Offline Mode: Showing parking data as of [Timestamp]."*
- **Permission Requests:** Standardized system dialogues for Location Services and Motion & Fitness permissions, accompanied by a UNT-branded explanation of the anonymity of the data.
- **Input Validation:** The Admin Dashboard will use red inline text for invalid entries (e.g., entering a negative number for "Total Spaces").

3.2 Hardware Interfaces

The EagleEye Parking system serves as a bridge between distributed mobile hardware and centralized parking enforcement infrastructure.

1. Mobile Device Sensors (Direct Interface)

The software interfaces with specific hardware sensors on user smartphones via the **ParkZen SDK** to automate "Parking Event" detection without manual user input.

- **Supported Device Types:**
  - **iOS:** iPhone 8 or newer (running iOS 15+).
  - **Android:** Devices with ARMv8 architecture or higher (running Android 10+).
- **Data & Control Interactions:**
  - **GPS/GNSS:** Used to establish geo-fence entry/exit events within 5 meters.
  - **Accelerometer & Gyroscope:** Detects "Automotive" vs. "Pedestrian" motion states to distinguish between parking a car and walking through a lot.
  - **Bluetooth Low Energy (BLE):** Scans for vehicle head-unit connectivity or nearby beacons to confirm proximity to the vehicle during a "Depart" event.
- **Communication Protocol:** Integration occurs via JSON over HTTPS. The FastAPI backend will handle asynchronous "handshaking" with the AIMS system to ensure that hardware count updates do not block the mobile user's request for occupancy data.

2. LPR Patrol Vehicles & AIMS Infrastructure (Indirect Interface)

The system leverages ground-truth data collected by university-managed hardware.

- **Hardware Source:** Mobile License Plate Recognition (LPR) cameras mounted on UNT Parking Services patrol vehicles.
- **Nature of Interaction:**
  - **Data Flow:** The LPR hardware scans plates and calculates vehicle counts per lot. This raw hardware data is stored in the **AIMS database**.
  - **Control Interaction:** EagleEye acts as a "Consumer." It issues **RESTful API calls** to the AIMS backend to retrieve the processed hardware counts.
- **Communication Protocol:** Integration occurs via **JSON over HTTPS**, utilizing OAuth 2.0 or API Key authentication for secure handshaking.

3. Network Infrastructure

- **Hardware:** University Wi-Fi (UNT-Eagles), Cellular (LTE/5G), and Cloud Server Nodes.
- **Requirements:**
    - **Mobile:** Minimum 3G connectivity required for occupancy updates.
    - **Backend:** The middleware must be hosted on hardware compatible with Docker and Python 3.10+ environments (utilizing Uvicorn or Gunicorn as needed). Deployment will be supported on either on-premise UNT servers or university-approved cloud instances.

3.3 Software Interfaces

The EagleEye Parking system acts as a central aggregator, managing data flow between three primary external ecosystems via a FastAPI middleware layer.

1. ParkZen Integration (SDK & Cloud API)

- **Component**: ParkZen Mobile SDK (iOS/Android) and ParkZen REST API.
- **Communication**: Authenticated RESTful API calls over HTTPS (TLS 1.3).
- **Inbound Data**: Real-time occupancy probabilities (0.0–1.0), lot status updates, and "Parking Event" confirmations.
- **Outbound Data**: Anonymized UUIDs and lot entrance/exit triggers.
- **Purpose:** To leverage crowdsourced sensor data for occupancy forecasting.

2. UNT AIMS Integration (Enforcement Data)

- **Component**: AIMS REST API (Version provided by UNT Parking Services).
- **Communication**: Secure JSON-based API calls using FastAPI's httpx library for asynchronous polling.
- **Inbound Data**: Physical vehicle counts per lot, timestamp of last LPR scan, and permit-to-lot authorization rules.
- **Outbound Data**: Authentication tokens and specific Lot_ID queries.
- **Purpose**: To provide "Ground Truth" data to calibrate the predictive model.

3. Database Management (PostgreSQL)

- **Component:** PostgreSQL 14+ managed via SQLAlchemy (ORM) or Tortoise-ORM.
- **Data Items**: Parking lot inventory (GPS polygons), historical occupancy logs (aggregated), hazard reports, and administrative notification logs.
- **Implementation Constraint**: Zero Direct Access. No mobile client or external service may query the database directly. All interactions must pass through the FastAPI backend via an authenticated Data Access Layer (DAL).

4. Push Notification Services (APNS & FCM)

- **Component:** Apple Push Notification Service (APNS) for iOS and Firebase Cloud Messaging (FCM) for Android.
- **Purpose**: To broadcast "Game Day" towing alerts, hazard notifications, and citation warnings (PR-004).
- **Communication**: Asynchronous message queuing via the backend to ensure high delivery speed.

5. Integrated Web Views (AIMS Permit Portal)

- **Component**: Embedded Web View (WKWebView for iOS / WebView for Android).
- **Purpose**: To provide a seamless "Purchase Permit" method within the UNT Go tile that redirects to the external AIMS payment site.
- **Data Shared**: The system may pass an anonymous lot_id to the portal to pre-select the relevant permit page, but no personal identifiable information is shared during this redirect.

3.4 Communications Interfaces

The EagleEye Parking system relies on high-frequency, asynchronous communication between mobile clients, the FastAPI middleware, and third-party vendor clouds.

1. Network Protocols and Standards

- **Primary Protocol**: All communication between the mobile client, the administration dashboard, and the FastAPI backend must occur over HTTPS (Hypertext Transfer Protocol Secure) using TLS 1.3.
- **API Architecture:** The system utilizes a REST (Representational State Transfer) architecture.
- **Data Formatting:** All messages, including lot status, permit rules, and hazard reports, will be formatted as JSON (JavaScript Object Notation).

2. Mobile-to-Server Communication

- **Synchronization Mechanism:** The mobile client will use a Polling and Webhook hybrid approach.
    - **Polling:** The client requests occupancy updates at a defined interval (e.g., every 60 seconds) or upon a "Map Refresh" trigger.
    - **Asynchronous Notifications:** Critical alerts (Towing, Citations, Hazards) will be pushed via Firebase Cloud Messaging (FCM) or Apple Push Notification Service (APNS) to ensure near-real-time delivery without constant polling.
- **Security:** All API requests from the mobile app must include a JWT (JSON Web Token) in the Authorization header to verify the user's session.

3. Server-to-External API Communication

- **ParkZen & AIMS Integration**: The FastAPI backend acts as a client to the ParkZen and AIMS REST APIs.
- **Synchronization:** * ParkZen: Near-real-time ingestion of parking events.
    - **AIMS:** Asynchronous scheduled tasks will poll the AIMS LPR counts every 5–10 minutes to synchronize the "Ground Truth" data.
- **Formatting:** Requests to AIMS will utilize OAuth 2.0 or API Keys as required by UNT IT infrastructure.

4. Electronic Forms & User Input

- **Validation:** All user-submitted data (e.g., reporting a hazard or updating a permit type) will be transmitted via JSON-encoded POST requests.

- **Integrity:** Input is validated on the server side using Pydantic models (native to FastAPI) to prevent SQL injection or malformed data from reaching the PostgreSQL database.

4. System Features

This section describes the system features of EagleEye Parking, organized by functional area. Each feature includes a description, priority level, stimulus/response sequences, and detailed functional requirements. Requirements are uniquely identified and traceable to stakeholder needs identified during the February 3, 2026, stakeholder meeting and subsequent communications.

4.1 Real-Time Parking Occupancy Display

4.1.1 Description and Priority

The system shall display real-time parking lot occupancy information using a color-coded map interface. Occupancy data is sourced from the ParkZen crowdsourcing engine and supplemented by AIMS license plate reader (LPR) vehicle counts. Occupancy is represented as a probability of finding parking, displayed using a green/yellow/orange/red color scheme. This is the core feature of the application and the primary value proposition for end users.

The ParkZen SDK runs on the user's device and uses phone sensors (GPS, Bluetooth, accelerometer) to detect parking events anonymously. For the SDK to function, the app must request location and motion/fitness permissions from the user upon first launch. No user identity or route data is collected; all sensor analysis is performed locally on the device and only anonymized event data is sent to the ParkZen cloud. The system must follow FERPA regulations and UNT data security policies.

Priority: High (Benefit: 9, Penalty: 9, Cost: 5, Risk: 4)

> ***User Story:*** *As a student, I want to see real-time parking lot occupancy (color-coded) so that I can find available parking quickly.*

4.1.2 Stimulus/Response Sequences

Step	User Action	System Response
1	User opens the EagleEye Parking app for the first time	System displays a clear, transparent permission request explaining what data is collected and how it is used, emphasizing that data is anonymized and no identity or routes are tracked
2	User grants location and motion/fitness permissions	ParkZen SDK begins local sensor analysis to detect parking events; the app loads the campus map with occupancy data
3	User denies permissions	App still loads the campus map with occupancy data from other sources (AIMS, other opted-in users); the user can use the app normally but does not contribute crowdsourced data
4	User opens the app or navigates to the parking section (subsequent launches)	System loads the campus map with all parking lots displayed with color-coded occupancy overlays
5	System retrieves current occupancy data from ParkZen API and AIMS API	Map renders each lot with a color overlay (green, yellow, orange, or red) based on occupancy percentage
6	User taps on a specific parking lot	System displays a detail panel showing lot name, estimated available spaces, permit types

		accepted, last updated timestamp, and lot-specific information
7	Occupancy data updates in the background	Map colors refresh automatically without requiring user interaction
8	ParkZen or AIMS data is temporarily unavailable	System displays a stale-data warning indicator with the timestamp of the last successful update

4.1.3 Functional Requirements

Req ID	Description	Priority
REQ-1.1	The system shall retrieve parking occupancy data from the ParkZen API at a configurable refresh interval (default: 60 seconds).	High
REQ-1.2	The system will visualize parking availability using three dynamic color levels: Green (0%–85%) for available, Orange (85%–95%) for nearing capacity, and Red (95%–100%) for lots that are full or closed.	High
REQ-1.3	The system shall display an interactive campus map showing all parking lots with color-coded occupancy overlays.	High
REQ-1.4	The system shall allow users to tap a parking lot to view detailed information including: lot name, estimated available spaces, accepted permit types, and last-updated timestamp.	High
REQ-1.5	The system shall display a data freshness indicator showing when occupancy data was last updated so users can gauge reliability.	High
REQ-1.6	The system shall display a stale-data warning if occupancy data has not been updated within a configurable threshold (default: 5 minutes).	Medium
REQ-1.7	The system shall support filtering the map view by permit type so users see only lots relevant to their permit.	Medium
REQ-1.8	The system shall request location services and motion/fitness activity permissions from the user on first launch with a clear explanation of data usage and privacy protections.	High
REQ-1.9	The system shall remain fully functional for users who decline permissions, displaying occupancy data sourced from AIMS and other opted-in users.	High
REQ-1.10	The system shall follow FERPA regulations and UNT data security policies. No personally identifiable information shall be collected, stored, or transmitted by the application.	High

4.2 AIMS/LPR Data Integration

4.2.1 Description and Priority

The system shall integrate with UNT's AIMS (license plate reader) system to ingest vehicle count data from LPR patrol vehicles. This data supplements ParkZen crowdsourced occupancy estimates with ground-truth vehicle counts collected during enforcement patrols. ParkZen is already coordinating with AIMS for API credentials, and the capstone team will assist with this integration.

Priority: High (Benefit: 8, Penalty: 7, Cost: 6, Risk: 5)

> ***User Story:*** *As a parking administrator, I want to integrate LPR/AIMS vehicle count data into the system so that occupancy estimates are more accurate.*

4.2.2 Stimulus/Response Sequences

Step	User Action	System Response
1	AIMS LPR patrol vehicle completes a scan of a parking lot	AIMS system records vehicle count and publishes data via API
2	EagleEye backend polls the AIMS API on a scheduled interval	System retrieves the latest vehicle count data for scanned lots
3	Backend processes AIMS data and merges with ParkZen occupancy estimates	The combined occupancy estimate is computed and stored
4	AIMS API is unreachable or returns an error	System logs the failure, continues using ParkZen-only estimates, and retries on next interval

4.2.3 Functional Requirements

Req ID	Description	Priority
REQ-2.1	The system shall retrieve vehicle count data from the AIMS API using authenticated REST calls.	High
REQ-2.2	The system shall map AIMS lot identifiers to the internal parking inventory so that vehicle counts are attributed to the correct lots.	High
REQ-2.3	The system shall merge AIMS vehicle count data with ParkZen crowdsourced estimates using a weighted algorithm to produce a combined occupancy value.	High
REQ-2.4	The system shall log all AIMS API interactions (requests, responses, errors) for debugging and audit purposes.	Medium
REQ-2.5	The system shall gracefully handle AIMS API outages by falling back to ParkZen-only occupancy data without user-facing errors.	High

4.3 Parking Inventory Management

4.3.1 Description and Priority

The system shall maintain a comprehensive parking inventory containing all UNT parking lots, their total space counts, geographic boundaries, and accepted permit types. This inventory serves as the foundational data layer for occupancy calculations and map display. The initial inventory will be loaded from a spreadsheet provided by UNT Parking Services and maintained in the ParkZen dashboard. The permit-area permission matrix defines which permits are valid in which lot types during daytime (Mon-Fri 7 AM-5:30 PM) and nighttime (Mon-Fri 5:30 PM-7 AM, all day Saturday and Sunday) enforcement periods.

Priority: High (Benefit: 9, Penalty: 8, Cost: 3, Risk: 2)

4.3.2 Stimulus/Response Sequences

Step	User Action	System Response
1	Administrator uploads or updates parking inventory data (lot names, space counts, permit types, boundaries)	System validates the data and updates the internal inventory database
2	User opens the parking map	System loads lot locations, boundaries, and metadata from the inventory
3	Administrator modifies a lot (adds spaces, changes permit types, updates boundaries)	System updates the inventory and reflects changes on the next map refresh
4	Inventory data contains validation errors (missing fields, invalid coordinates)	System rejects the update and displays specific error messages

4.3.3 Functional Requirements

Req ID	Description	Priority
REQ-3.1	The system shall store parking inventory data, including: lot name/ID, geographic location (coordinates/boundaries), total number of spaces, and accepted permit types.	High
REQ-3.2	The system shall support the following permit types as defined by UNT Parking Services: AA (All Access), A (A-Reserved), AH (Highland Reserved), E (Eagle), FS (Faculty/Staff), HD (Handicap), HSF, HSG, HSR, MC (Motorcycle), R (Remote), RM, RR (Restricted Reserved), SV (Service Vehicle), UCG, VIS (Visitor), Night, Evening, Teaching Fellow, DAYTEMP, and Eagle Fall.	High
REQ-3.3	The system shall enforce the permit-area permission matrix, which defines which permits are valid in which lot types during daytime enforcement (Mon-Fri 7:00 AM - 5:30 PM) and nighttime enforcement (Mon-Fri 5:30 PM - 7:00 AM and all day Sat/Sun).	High
REQ-3.4	The system shall support the 'park down' rule: higher-tier permits (e.g., A) can park in lower-tier lots (e.g., Eagle, FS), but A and RR permits are restricted to their assigned lots and cannot use alternate A or RR lots, respectively.	High
REQ-3.5	The system shall support initial bulk import of parking inventory from a spreadsheet format (lot name, location, total spaces, permit types, optional map boundaries).	Medium
REQ-3.6	The system shall validate inventory data on import, rejecting entries with missing required fields and reporting specific validation errors.	Medium

4.4 Push Notifications and Alerts

4.4.1 Description and Priority

The system shall support push notifications to inform users about parking-related events, including citation warnings, game day towing restrictions, lot closures, and hazard alerts. Game day towing is a significant issue at UNT, with 100-150 cars historically towed per event. Proactive notification of freshmen and new students about game day restrictions is a key stakeholder requirement.

Priority: High (Benefit: 9, Penalty: 8, Cost: 5, Risk: 3)

> ***User Story:*** *As a student, I want to receive push notifications about game day towing restrictions so that I don't get my car towed.*

4.4.2 Stimulus/Response Sequences

Step	User Action	System Response
1	Parking administrator creates a notification (citation warning, game day alert, closure notice)	System queues the notification for delivery to targeted users
2	System identifies the target audience based on the notification type and the user permit/lot associations	Push notification is delivered to the matching users' devices
3	User receives and opens a push notification	App navigates to the relevant lot or alert detail view
4	The game day event is scheduled in the system	The system automatically sends towing restriction warnings to users who park in affected lots at a configurable lead time before the event
5	User has notifications disabled on their device	System logs the delivery failure; notification content is available in-app when the user next opens the application

4.4.3 Functional Requirements

Req ID	Description	Priority
REQ-4.1	The system shall support push notifications for the following event types: citation warnings, game day towing restrictions, lot closures, hazard alerts, and construction/event notices.	High
REQ-4.2	The system shall allow parking administrators to create and send push notifications to all users or to users filtered by permit type or lot assignment.	High
REQ-4.3	The system shall support scheduled notifications that are automatically sent at a configurable time before an event (e.g., game day towing warning sent 24 hours before kickoff).	Medium
REQ-4.4	The system shall store all notifications in an in-app notification history so users who missed a push notification can view it later.	Medium
REQ-4.5	The system shall support citation warning notifications that alert users when they may be at risk of receiving a parking citation based on their permit type and current lot.	Low

4.5 Hazard and Closure Reporting

4.5.1 Description and Priority

The system shall allow both students and parking staff to report real-time hazards (e.g., ice, flooding, debris) and lot closures. Other ParkZen-integrated universities have used this feature heavily during icy conditions. Parking staff can also push closure alerts for construction, events, or weather emergencies. Lots can be marked as fully closed, partially closed, or scheduled for future closure.

Priority: Medium (Benefit: 7, Penalty: 5, Cost: 4, Risk: 3)

> ***User Story:*** *As a student, I want to report hazards (like ice) in parking lots so that other students can avoid dangerous areas.*

4.5.2 Stimulus/Response Sequences

Step	User Action	System Response
1	Student reports a hazard in a parking lot (e.g., ice, flooding)	System records the hazard report with lot ID, hazard type, timestamp, and optional description
2	Multiple students report the same hazard in the same lot	System aggregates reports and escalates visibility (e.g., hazard icon appears on map)
3	Parking administrator marks a lot as closed or partially closed	System updates the map to show the lot as closed (different color/striping) and sends notifications to affected users
4	Administrator schedules a future lot closure (e.g., game day, construction)	System records the scheduled closure and triggers automated notifications at the configured lead time
5	Administrator clears a hazard or reopens a closed lot	The system removes the hazard indicator or closure marker and updates the map

4.5.3 Functional Requirements

Req ID	Description	Priority
REQ-5.1	The system shall allow authenticated users to submit hazard reports for any parking lot, specifying the hazard type (ice, flooding, debris, lighting, etc.) and an optional text description.	Medium
REQ-5.2	The system shall display active hazard reports on the parking map with a visual indicator on the affected lot.	Medium
REQ-5.3	The system shall allow parking administrators to mark lots as fully closed, partially closed (with a percentage), or scheduled for closure.	High
REQ-5.4	The system shall display closed lots with distinct visual styling (striping or grayed out) on the map, clearly distinguishable from occupancy colors.	Medium
REQ-5.5	The system shall allow parking administrators to clear hazard reports and reopen closed lots.	Medium
REQ-5.6	The system shall automatically expire hazard reports after a configurable time period (default: 24 hours) if not manually cleared.	Low

4.6 Permit Information and Park-Down Rules Display

4.6.1 Description and Priority

The system shall display permit-specific parking information to help users understand where their permit is valid. UNT supports a complex permit hierarchy with 'park down' rules (e.g., A permits can park in Eagle or FS spaces but not in other A-assigned lots). Mixed permit environments are supported, including section-level restrictions within lots. Lot enforcement hours differ: most lots are enforced 24/7, while Lot 60 is enforced Mon-Fri 7:00 AM - 5:30 PM. FIR permits are allowed only in FIR spaces or Remote areas.

Priority: Medium (Benefit: 7, Penalty: 6, Cost: 3, Risk: 2)

> ***User Story:*** *As a student, I want to see which lots my permit allows me to park in so that I avoid parking violations.*

4.6.2 Stimulus/Response Sequences

Step	User Action	System Response
1	User selects their permit type in the app, or it is detected from their profile	System filters the map to highlight lots where the user's permit is valid for the current time of day
2	User views a lot detail for a lot with mixed permit sections	The system displays section-level permit information with text explaining where each permit type can park within the lot
3	Enforcement period changes (e.g., daytime to nighttime)	System updates which lots as valid for the user's permit type based on the day/night enforcement matrix
4	User has a lot-specific permit (A or RR) and views the map	System highlights only the user's assigned lot plus valid park-down options; other A or RR lots are not highlighted

4.6.3 Functional Requirements

Req ID	Description	Priority
REQ-6.1	The system shall allow users to select or store their permit type to enable personalized lot filtering.	Medium
REQ-6.2	The system shall filter the parking map based on the user's permit type, highlighting lots where parking is valid during the current enforcement period (daytime or nighttime).	Medium
REQ-6.3	The system shall correctly implement the 'park down' hierarchy, showing lower-tier valid lots for higher-tier permits.	Medium
REQ-6.4	The system shall display lot-specific text information for lots with mixed permit sections, explaining which areas accept which permit types.	Low
REQ-6.5	The system shall distinguish between daytime enforcement (Mon-Fri 7:00 AM - 5:30 PM) and nighttime enforcement (Mon-Fri 5:30 PM - 7:00 AM, all day Saturday and Sunday) when filtering lots.	Medium
REQ-6.6	The system shall display Premium Parking (visitor hourly parking) availability in designated visitor lots and signed spaces within other lots, showing it as available 24/7.	Low

4.7 Administrative Dashboard and Analytics

4.7.1 Description and Priority

The system shall provide parking administrators with a dashboard for managing parking operations, viewing occupancy analytics, and configuring system settings. The ParkZen dashboard provides analytics and forecasting capabilities. Administrators can view historical usage patterns, manage lot configurations, push alerts, and generate reports to support data-driven transportation planning.

Priority: Medium (Benefit: 7, Penalty: 5, Cost: 5, Risk: 3)

> ***User Story:*** *As a parking administrator, I want to view occupancy analytics and forecasting data so that I can make informed decisions about parking resource allocation.*

4.7.2 Stimulus/Response Sequences

Step	User Action	System Response
1	Administrator logs into the ParkZen dashboard	System authenticates the administrator and displays the main dashboard with a real-time campus-wide occupancy overview
2	The administrator selects a specific lot	The system displays detailed occupancy history, peak usage times, and trend data for that lot
3	The administrator creates or modifies a lot closure, hazard alert, or push notification	System validates the input, saves the configuration, and triggers relevant notifications
4	The administrator requests an analytics report for a date range	The system generates a report showing occupancy patterns, peak times, and utilization rates

4.7.3 Functional Requirements

Req ID	Description	Priority
REQ-7.1	The system shall provide an administrative dashboard accessible to authorized UNT Parking Services staff.	Medium
REQ-7.2	The dashboard shall display real-time occupancy data for all campus parking lots in a summary view.	Medium
REQ-7.3	The dashboard shall display historical occupancy data and trend analytics for individual lots over configurable date ranges.	Low
REQ-7.4	The dashboard shall allow administrators to manage lot configurations (closures, permit changes, special event settings).	Medium
REQ-7.5	The dashboard shall provide role-based access control, restricting administrative functions to authorized personnel.	Medium

4.8 Permit Purchase Integration

4.8.1 Description and Priority

The system shall integrate a permit purchase pathway within the application. Currently, permit purchases link out to the AIMS website. Starting June 2026, UNT transitions to fully digital permits (no physical stickers), making in-app permit access more critical. This feature does not require location permissions and serves as an early engagement point to increase app adoption before crowdsourced data collection begins.

Priority: Medium (Benefit: 6, Penalty: 4, Cost: 2, Risk: 2)

> ***User Story:*** *As a student, I want to purchase my parking permit through the app so that I don't have to visit a separate website.*

4.8.2 Stimulus/Response Sequences

Step	User Action	System Response
1	User navigates to the permit section of the app	System displays permit purchase options and links
2	The user selects 'Purchase Permit.'	System opens the AIMS permit purchase page (via embedded webview or external link)
3	User completes the permit purchase on the AIMS website	User returns to the app; permit type may be associated with the user profile for personalized lot filtering

4.8.3 Functional Requirements

Req ID	Description	Priority
REQ-8.1	The system shall provide a permit purchase pathway that links to the AIMS permit purchase website.	Medium
REQ-8.2	The system shall display information about available permit types and their associated parking areas before directing users to the purchase page.	Low
REQ-8.3	The system shall support future integration with digital permit verification once UNT transitions to digital-only permits in June 2026.	Low

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Requirements	Descriptions
PR-Latency	The system must reflect a change in lot occupancy (green to yellow,...) within 120 seconds of a “Parked” or “Departed” event being detected Rationale: to prevent drivers from navigating to a lot that has already filled up during their transit.
PR-Concurrency	The middleware and dashboard must support at least 500 concurrent API calls during peak class-change intervals without exceeding a 2-second response time.
PR-Availability	The ParkZen dashboard and data feeds must maintain 99.5% uptime during university operating hours.

5.2 Safety Requirements

Requirements	Descriptions
SR-Hazard reporting	The system must prioritize the delivery of “Safety Alerts” (e.g., ice hazards reported by admins) over standard occupancy updates. These must be delivered to active app users within 30 seconds of publication.
SR-Distraction Mitigation	The UI design must follow “Hands-free” safety principles. No complex text entry should be required while the app detects that the device is moving at speeds over 5mph.
SR-Towing Prevention	On “game days”, the system must trigger a high-priority push notification if a user attempts to park in a lot scheduled for towing within the next 4 hours.

5.3 Security Requirements

Requirements	Descriptions
SER-Anonymization	No personally identifiable information, such as student ID, Name, or License Plate numbers, shall be stored in the ParkZen cloud. All data must be associated with an Anonymous UUID.
SER-Data Residency	In compliance with UNT policy, any middleware processing sensitive system data must reside within UNT-approved hosting or the local device
SER-FERPA	The student team is strictly prohibited from accessing the UNT Student

Compliance	Information System. Integration must be limited to anonymous parking tokens to ensure zero exposure to educational records.
SER-Encryption	All data in transit between the AIMS API, the Middleware, and ParkZen must be encrypted using TLS 1.2 or higher

5.4 Software Quality Attributes

Quality Characteristics	Descriptions
Usability (Priority 1)	A first-time user should be able to identify the nearest “Green” lot within 5 seconds of opening the app.
Interoperability	The FastAPI middleware must be modular, utilizing Python's httpx or requests libraries to bridge the AIMS and ParkZen APIs.
Maintainability	Code must be documented using inline Type Hints (standard in FastAPI) and the automatically generated Swagger/Redoc endpoints. A UNT IT staff member should be able to verify API health via the /docs route.
Reliability	The system must handle “dirty data” (an LPR scan that fails or provides a partial plate,...) by discarding the outlier rather than crashing the occupancy calculation.

5.5 Business Rules

Rules	Descriptions
Access control	Only UNT Parking Administrators have the authority to manually override lot status (like marking the lot as “closed” for construction)
Priority rule	Live AIMS/LPR data shall always take precedence over “crowdsourced” predictions in the occupancy algorithm if the two sources conflict.
Permit hierarchy	The system must enforce the UNT Parking Policy: a student with an “Eagle” permit can park in Eagle lots, but cannot view “Faculty/Staff” lots as available options.
Reporting rule	Hazard reports submitted by students must be verified by an Admin or receive a minimum of 3 peer-confirmations before being broadcast as a “Verified Hazard” to the entire campus.

6. Other Requirements

6.1 Database Requirements

Requirements	Descriptions
DB-Spatial	The system must utilize PostGIS extensions within PostgreSQL to perform real-time spatial queries to validate if a device's coordinates fall within a lot's polygonal boundary.
DB-Archiving	To maintain performance, the occupancy_logs table must be partitioned **by month**. Logs older than 90 days must be aggregated into a historical_trends table and the raw entry deleted.
DB-Concurrency	The database must support **Asynchronous Connection Pooling** (via asyncpg) to allow the FastAPI backend to handle multiple simultaneous read/write requests during peak class-change hours.

6.2 Legal and Reuse Requirements

- **LEG-Data Sovereignty:** All parking data generated by the system is exclusive to UNT. No data may be stored outside of university-approved cloud regions.
- **REUSE-API:** The AIMS-to-ParkZen bridge logic must be written as a standalone Python module within the FastAPI app, allowing it to be reused if the university switches from a mobile app to physical digital signage.

Appendix A: Glossary

Term	Definition
AIMS	The proprietary parking management software used by UNT for permitting and LPR enforcement.
LPR	License Plate Recognition: high-speed cameras used to count vehicles and verify permits.
Geo-fence	A virtual geographic boundary, defined by GPS coordinates, that triggers a software response when a mobile device enters or leaves the area.
Ground Truth	The most accurate, verified data available (in this case, the LPR vehicle counts) is used to calibrate predictions.
JWT	JSON Web Token: a secure way to transmit information between parties as a JSON object.
PostGIS	An open-source software object that adds support for geographic objects to the PostgreSQL object-relational database.

Appendix B: Analysis Models

1. Entity-Relationship Diagram (ERD)

The database design is the most critical component, ensuring that every "Parking Event" is mapped to a specific "Lot" and "Permit Type."

2. Data Dictionary (subject to change)

- lots Table: Stores the lot_id, name, total_spaces, and a geometry column (PostGIS Polygon).
- permits Table: Stores permit_id, permit_name (e.g., Eagle, F/S), and the priority level.
- occupancy_data Table: Stores the current calculated status, merging parkzen_score and aims_count.
- hazard_reports Table: Stores hazard_type, location (Point), timestamp, and is_verified.

3. Data Flow Diagram (DFD)

This model shows how raw sensor data and AIMS API data move through the FastAPI middleware to update the PostgreSQL database.

Appendix C: To Be Determined List

TBD-01: Final **AIMS API Endpoints** and authentication keys (Pending Keitha/Parking Services).

TBD-02: Confirmation of **ModoLabs Sandbox** availability vs. Standalone Prototype path (Pending UNT CIO).
TBD-03: Specific **Permit Truth Matrix** spreadsheet (Pending Nikki/Parking Services).
TBD-04: Hosting environment credentials (Azure vs. On-Premise) for the **FastAPI/PostgreSQL** stack.